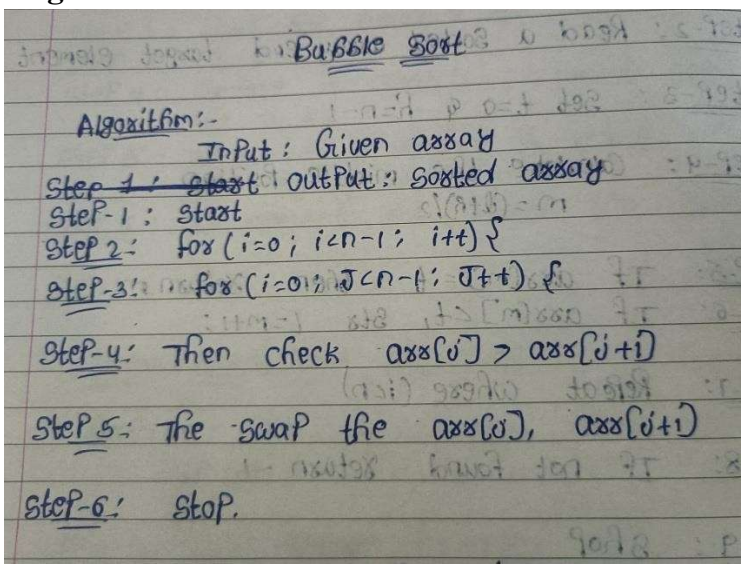
Practical 1

Aim:
To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Bubble sort

Algorithm:

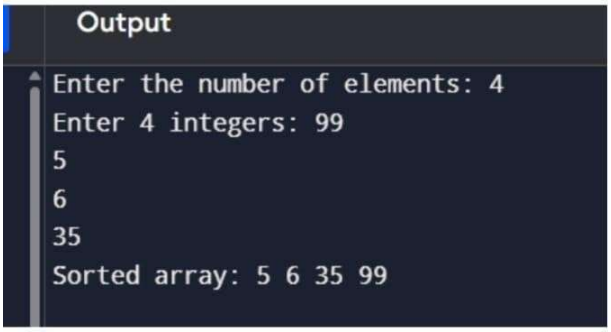


Code for bubble sort:

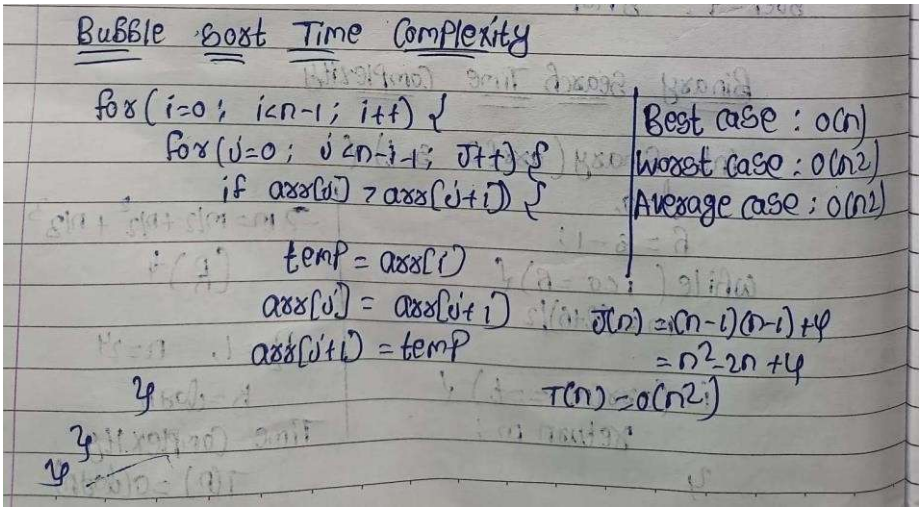
```
#include <iostream>
#include <vector>
using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    } }
```

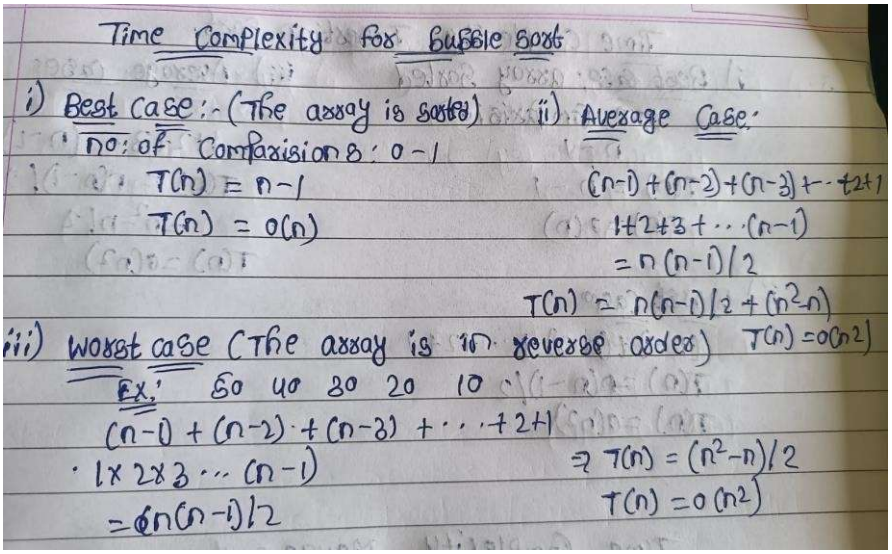
```
int main() {
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;
    vector<int> arr(size);
    cout << "Enter " << size << " integers: ";
    for (int i = 0; i < size; ++i) {
        cin >> arr[i];
    }
    bubbleSort(arr);
    cout << "Sorted array: ";
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output:



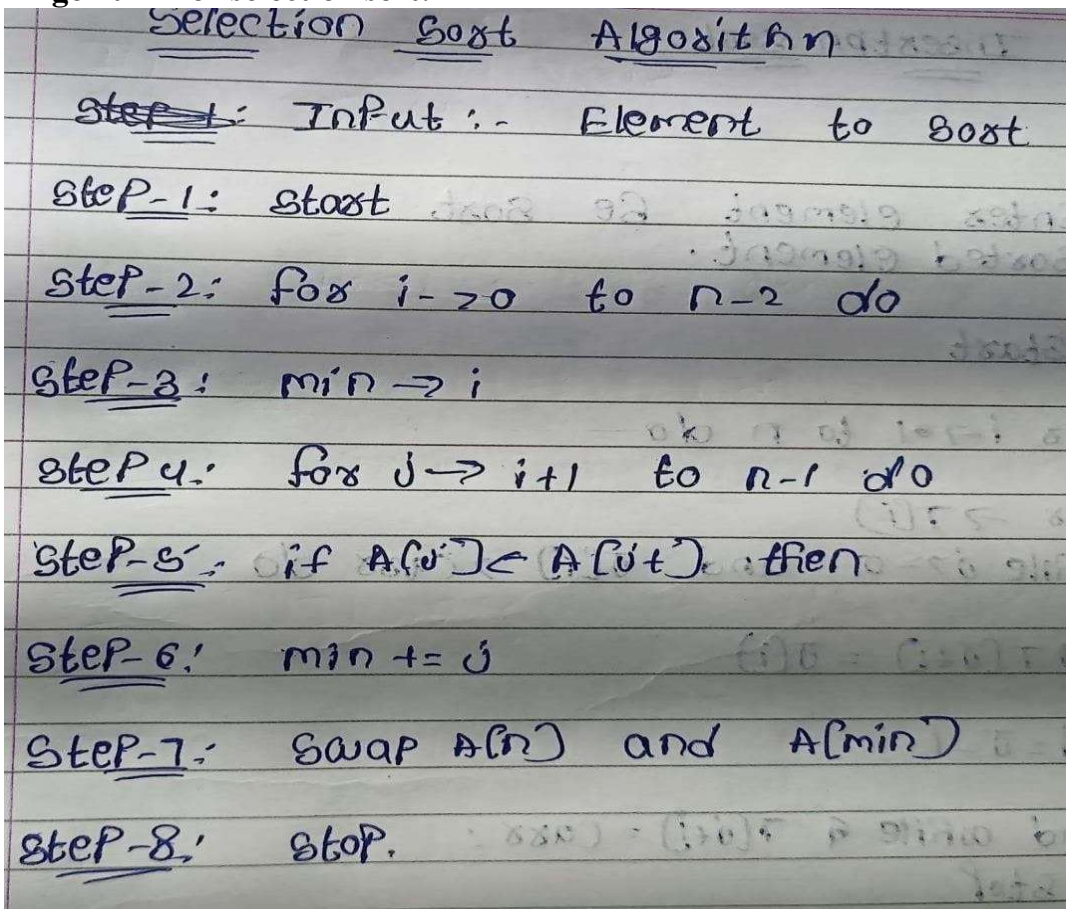
Time complexity:





Selection sort

Algorithm for selection sort:



Code for selection sort:

```
#include <iostream>
#include <vector>
void selectionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i], arr[minIndex]);
```

```
        }
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        return 0;
    }

    std::vector<int> arr(n);

std::cout << "Enter " << n << " integers: "; for
(int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

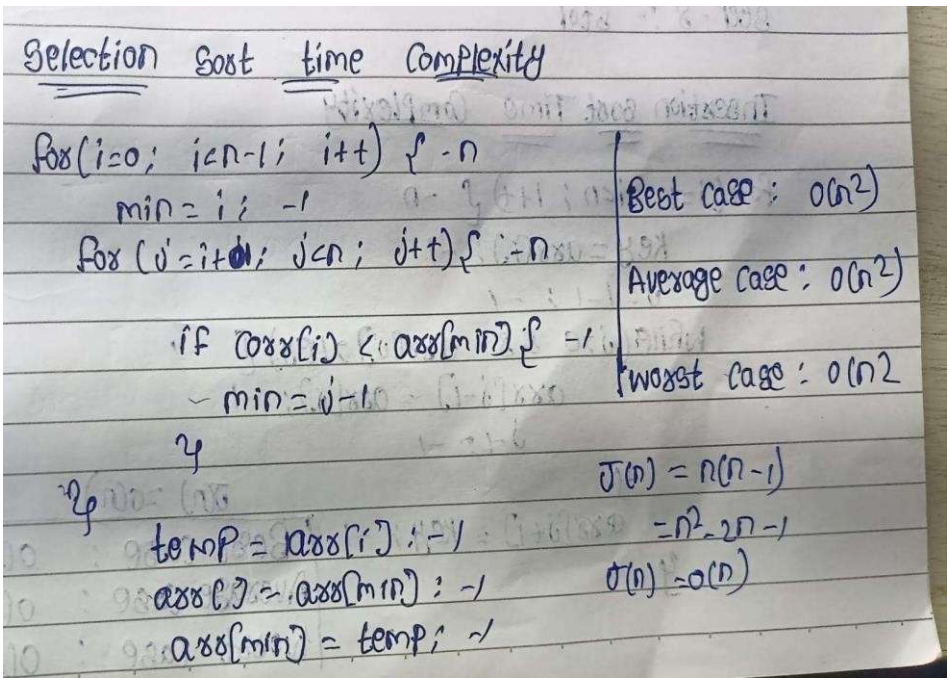
    selectionSort(arr);

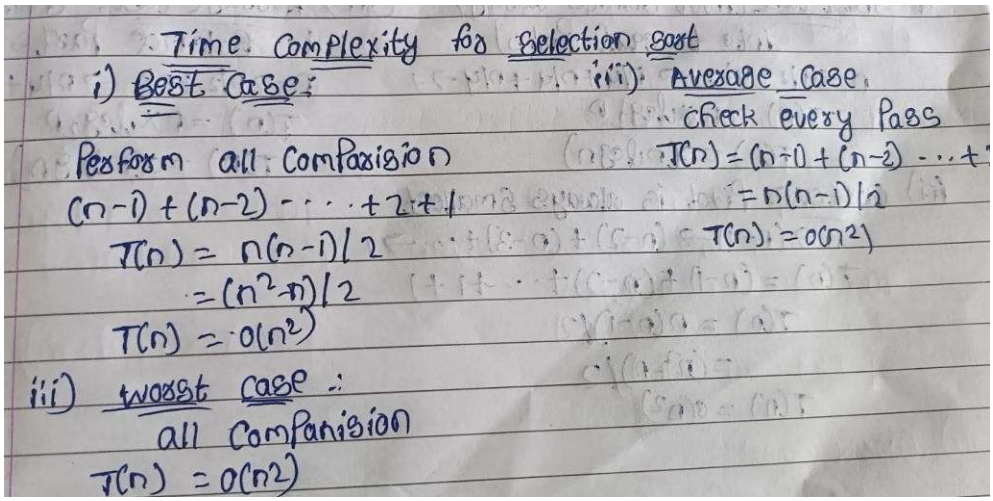
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 integers: 1
02
88
6
-8
Sorted array: -8 1 2 6 88
```

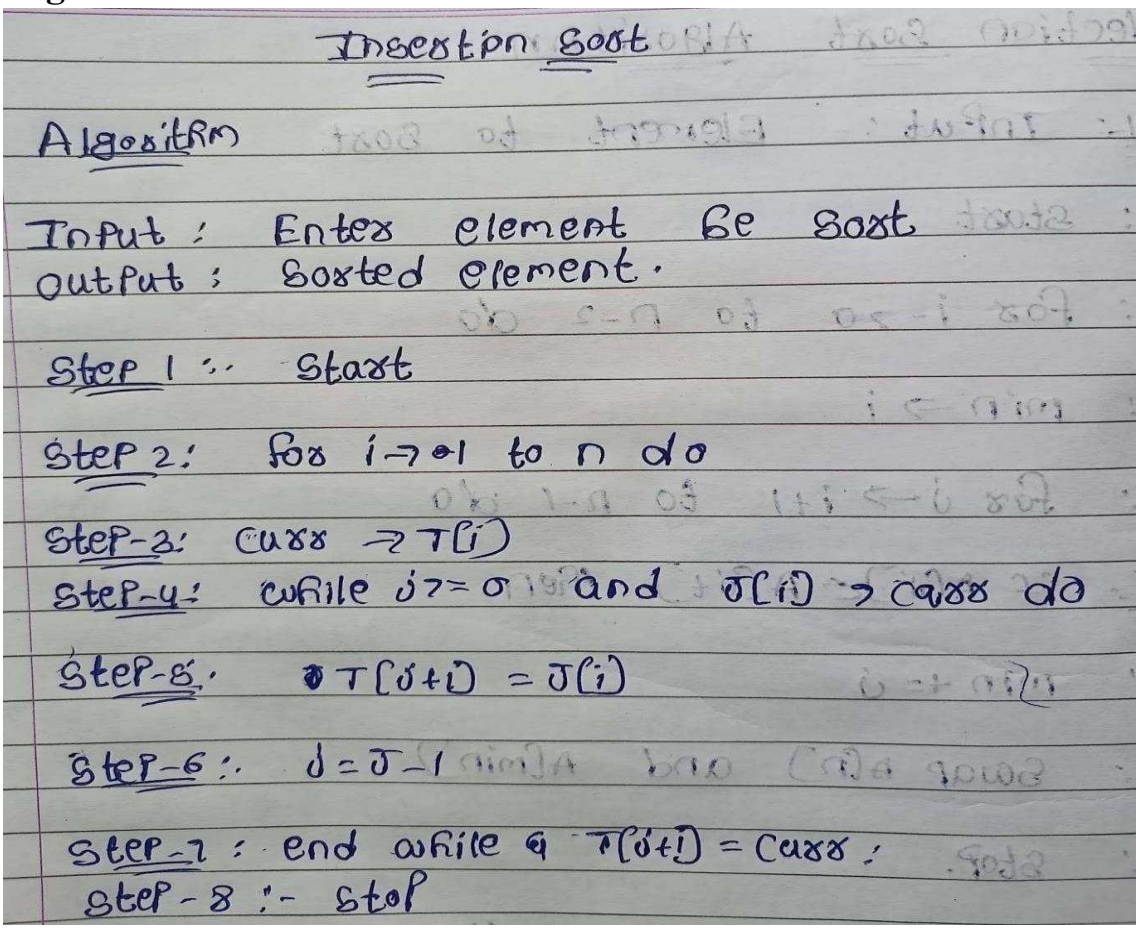
Time complexity:





Insertion sort

Algorithm:



Code:

```
#include <iostream>
#include <vector>
void insertionSort(std::vector<int>& arr) {
   int n = arr.size();
   for (int i = 1; i < n; ++i) {
      int key = arr[i];
      int j = i - 1;
      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j = j - 1;
      }
      arr[j + 1] = key;
   }
```

```
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        return 0;
    }

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";

    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

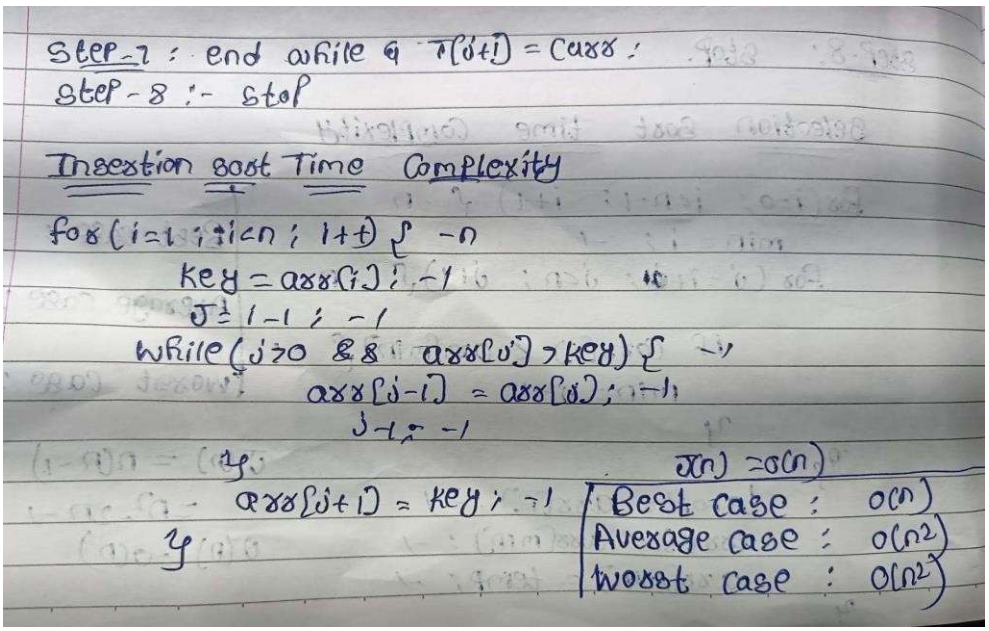
    insertionSort(arr);

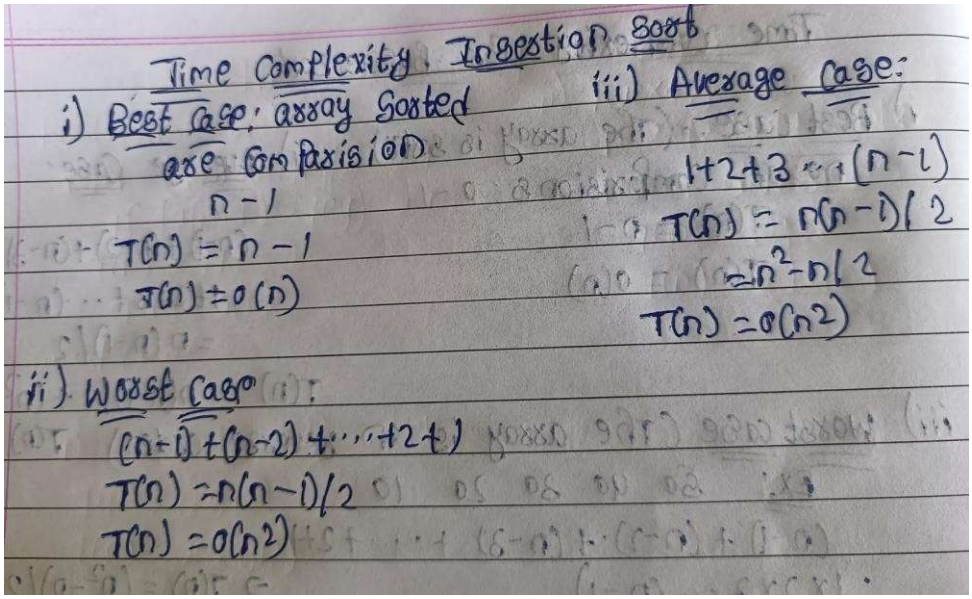
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 integers: 3
55
89
20
6
Sorted array: 3 6 20 55 89
```

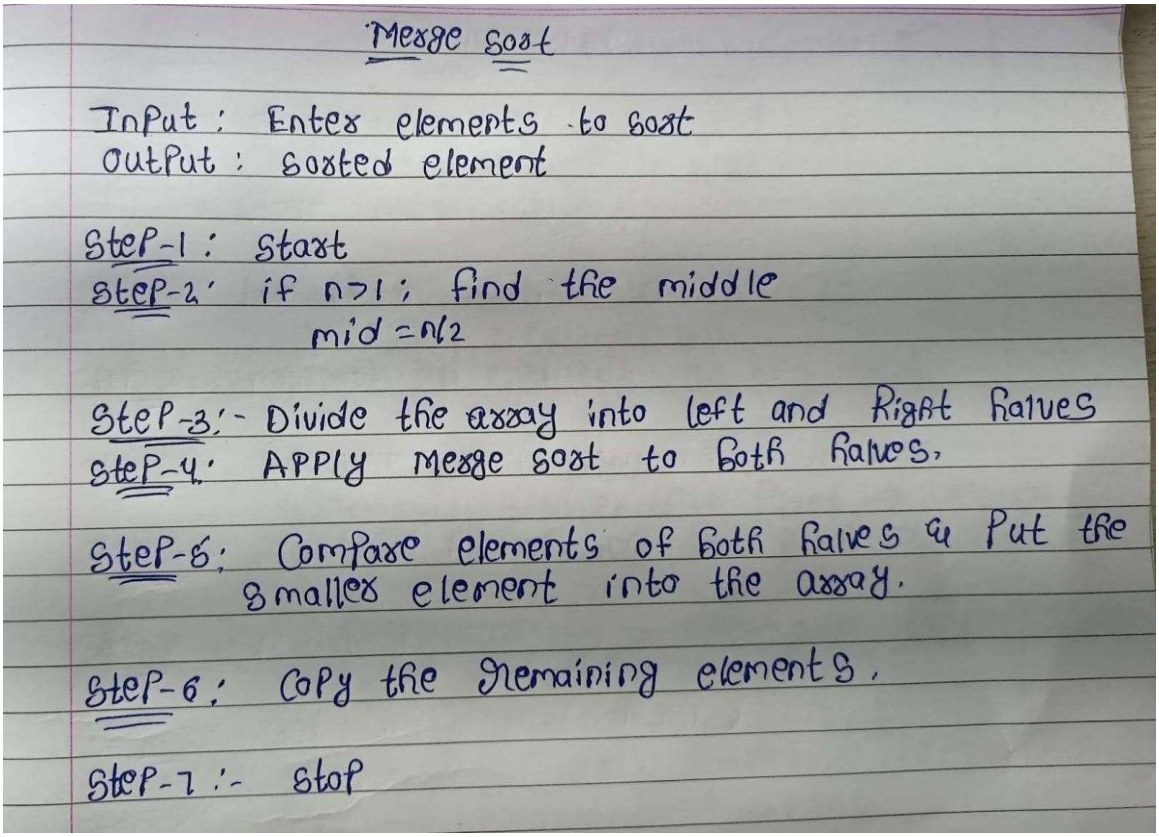
Time complexity:





Merge Sort:

Algorithm:



Code:

```
#include <iostream>
#include <vector>
void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - left;
    std::vector<int> L(n1);
    std::vector<int> R(n2);
    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }
int i = 0; int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            ++i;
        } else {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }
    while (i < n1) {
        arr[k] = L[i];
        ++i;
        ++k;
    }
    while (j < n2) {
        arr[k] = R[j];
        ++j;
        ++k;
    }
}

void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        return 0;
    }

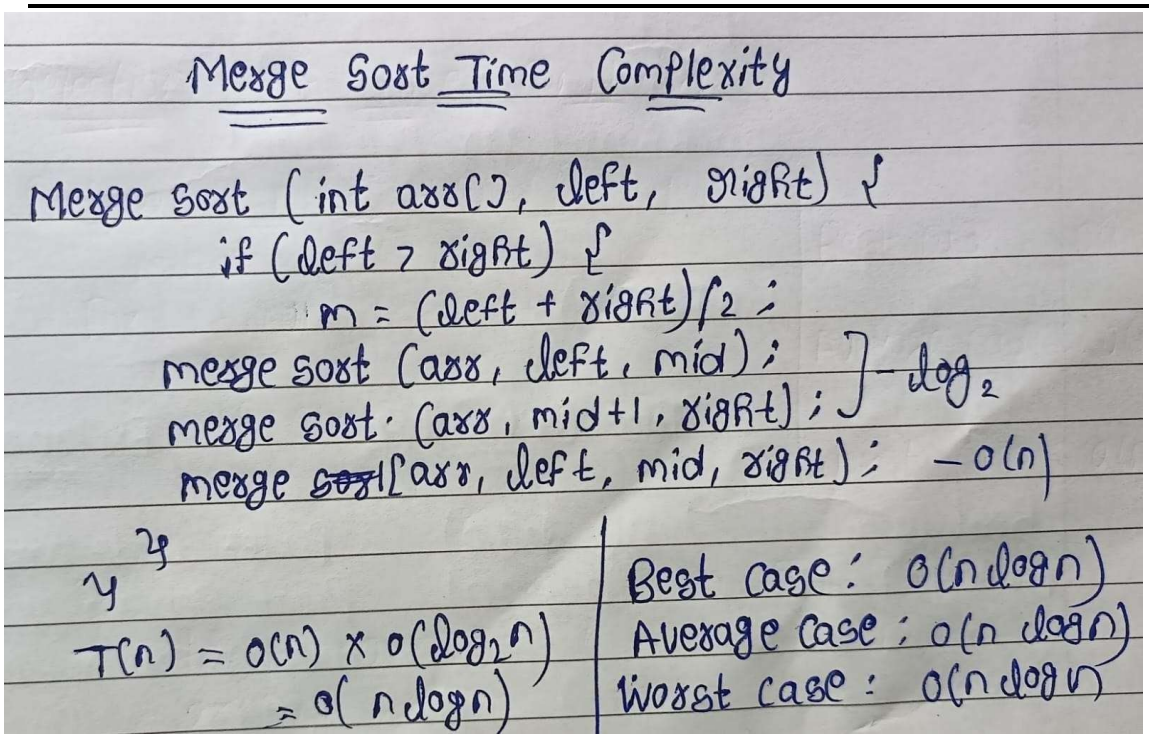
    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; ++i) {
```

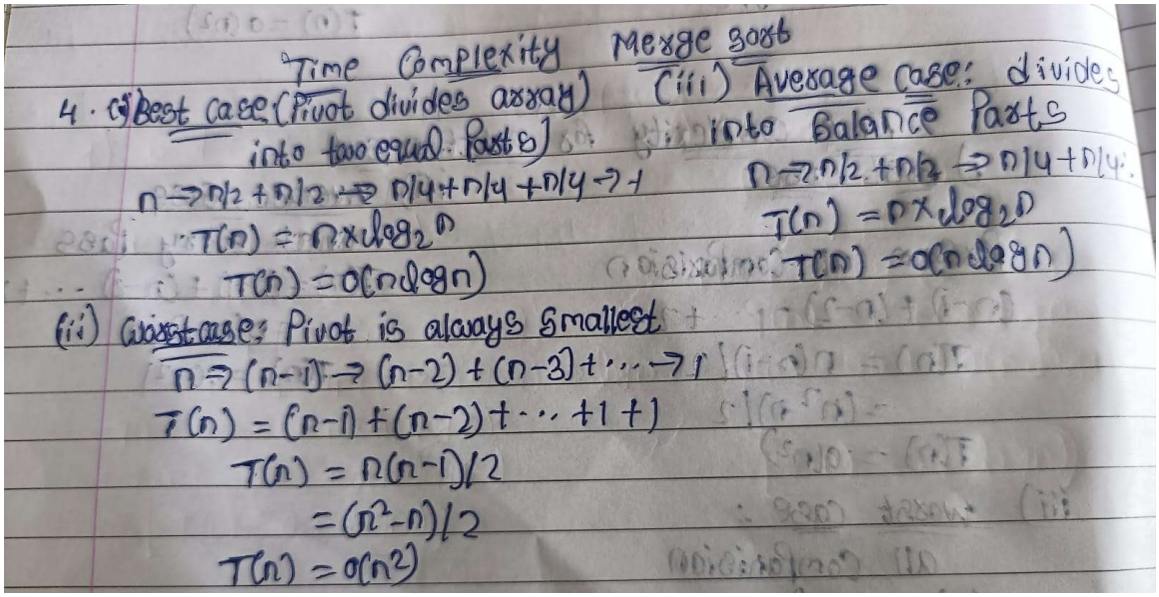
```
        std::cin >> arr[i];
    }
    mergeSort(arr, 0, n - 1);
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
std::cout << std::endl;
    return 0;
}
```

Output:

```
Output
Enter the number of elements: 5
Enter 5 integers: 66
4
7
45
5
Sorted array: 4 5 7 45 66
```

Time Complexity




Time Complexity Merge sort

4. (i) Best case (Pivot divides array into two equal Parts)

$n \to n/2 + n/2 \to n/4 + n/4 + n/4 \to 1$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

(ii) Worst case: Pivot is always Smallest

$n \to (n-1) \to (n-2) + (n-3) + \cdots \to 1$

$T(n) = (n-1) + (n-2) + \cdots + 1 + 1$

$T(n) = n(n-1)/2$

$= (n^2 - n)/2$

$T(n) = O(n^2)$

(iii) Average case: divides into Balance Parts

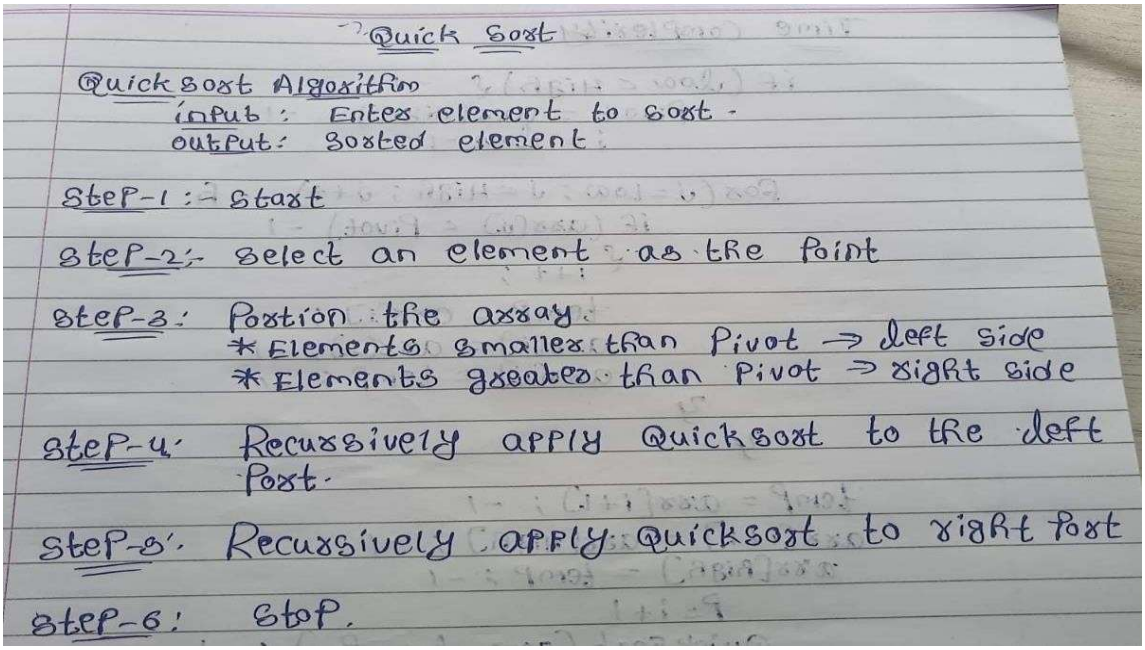
$n \to n/2 + n/2 \to n/4 + n/4$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

Quicksort:

Algorithm:



Code:

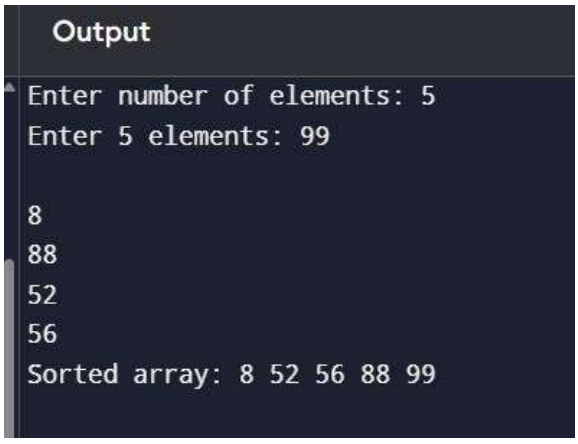
```
#include <iostream>
#include <vector>
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
```

```
}

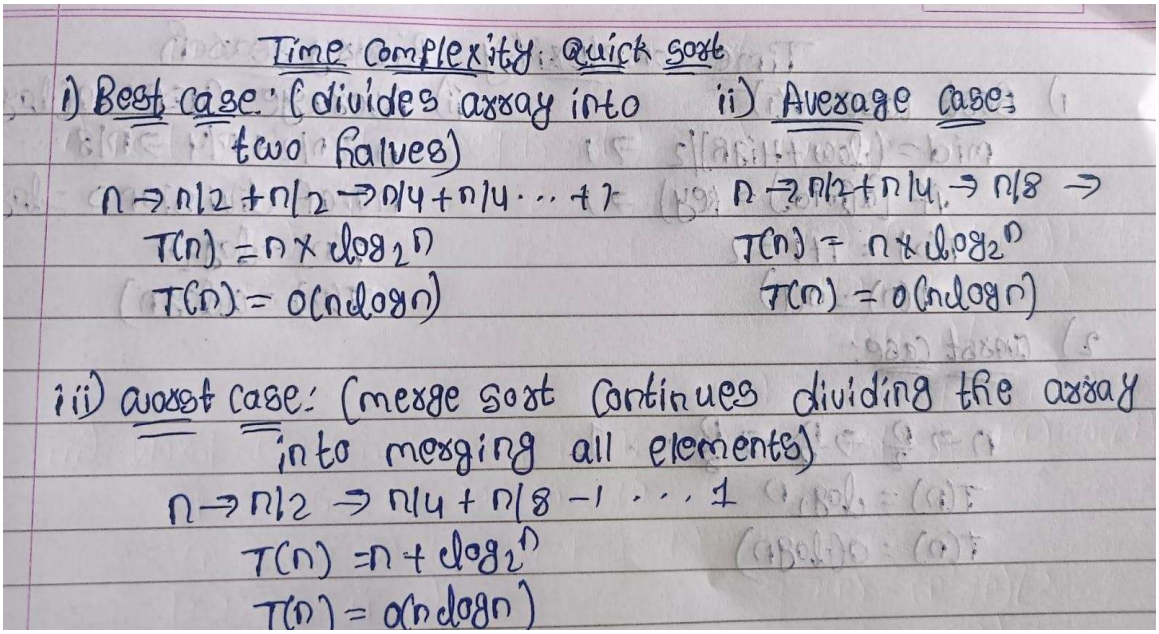
void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
    int n;
    std::cout << "Enter number of elements: ";
    std::cin >> n;

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        std::cin >> arr[i];
    }
    quickSort(arr, 0, n - 1);
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Output:



Time complexity


Time Complexity: Quick sort

i) Best case: (divides array into two halves)

$n \to n/2 + n/2 \to n/4 + n/4 \cdots + 1$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

ii) Average case:

$n \to n/2 + n/4 \to n/8 \to$

$T(n) = n \times \log_2 n$

$T(n) = O(n \log n)$

iii) worst case: (merge sort continues dividing the array into merging all elements)

$n \to n/2 \to n/4 + n/8 - 1 \cdots 1$

$T(n) = n + \log_2 n$

$T(n) = O(n \log n)$

Conclusion:

Thus , we studied and implemented different sorting algorithms.These five sorting algorithms demonstrate different approaches to sorting. Quick Sort and Merge Sort are generally the most efficient for large datasets, while Bubble, Selection, and Insertion Sort are simpler and useful for learning and small datasets.